Deep Learning

Assignment-05 (Report)

Generation Using Diffusion models

Course Name: Deep Learning

Course Code: CS508

Session: Spring 2026

Instructor: Dr. Mohsen Ali

Submitted by:

Mubashar Hussain

MSDS24031

Date of Submission: June 25, 2026

Contents

1 Github Repository URL **2**

2 Introduction **2**

3 Dataset **2**

4 Implementation Details **2**
4.1 Initial Implementation 2
4.2 Improvements During Development 3
4.3 Final Model 3

5 Experiments and Results **3**
5.1 Forward Diffusion 4
5.2 Training Performance 4
5.3 Generated Images 5
5.4 Reverse Diffusion Visualization 6

6 Conclusion **6**

1 Github Repository URL

- `https://github.com/mubashar-ds/Mubashar_MSDS24031_05`

2 Introduction

In this assignment, I implemented a simplified Denoising Diffusion Probabilistic Model (DDPM) using PyTorch. Complete pipeline was developed from scratch, including dataset preparation, forward diffusion, reverse diffusion, a U-Net based denoising model, training, and the image generation. My implementation focuses on understanding working principle of diffusion models rather than achieving state of the art image quality.

3 Dataset

Model was trained on small animal image dataset containing five classes:

- Cat, Lion, Tiger, Horse, Elephant

Before training, I resized all the images to $\mathbf{64}\times\mathbf{64}$ pixels and converted them into tensors. Pixel values were normalized to range of $[-1, 1]$ which is commonly used for diffusion models because Gaussian noise is added during the training. A sustom PyTorch dataset and dataloader were implemented to load images in mini batches during training.

4 Implementation Details

I made several improvements to obtain more stable training process and better image generation results, so my final implementation is based on configuration that produced lowest training loss and most consistent outputs.

4.1 Initial Implementation

My first implementation consisted of following components:

- Linear noise scheduler with 1000 diffusion steps.
- Forward diffusion for adding Gaussian noise.
- Reverse diffusion sampling.
- A lightweight U-Net consisting of encoder, bottleneck, and decoder blocks.
- A simple MLP based timestep embedding.
- Adam optimizer with Mean Squared Error (MSE) loss.

Model was initially trained for 30 epochs to verify that training pipeline and reverse diffusion process working correctly.

4.2 Improvements During Development

Then after obtaining first training results, several modifications were evalated to improve convergence and image quality.

Following improvements were retained in final implementation:

- Increased the number of training epochs from the initial experiments to 100 epochs.
- Saved the best model automatically based on the lowest training loss.
- Reduced learning rate to achieve smoother optimization.
- Added weight decay to improve generalization and reduce fluctuations during training.
- Added intermediate image saving during reverse diffusion to visualize denoising process.

Also I explored some additional architectural changes during experimentation, however these modifications did not improve generated results and therefore were not included in final version of model.

4.3 Final Model

Final implementation consists of:

- Image size: 64×64
- Linear noise schedule with 1000 diffusion steps
- Lightweight U-Net architecture
- Time embedding module
- Adam optimizer
- Mean Squared Error (MSE) loss
- Weight decay
- Best model checkpoint saving
- Reverse diffusion visualization during inference

These modifications produced stable convergence during training and enabled model to generate structured images from random Gaussian noise.

5 Experiments and Results

I performed several experiments during this assignment to improve performance of diffusion model.

5.1 Forward Diffusion

Forward diffusion process was verified by adding Gaussian noise to an image at different timesteps. As expected the image gradually lost its visual information and became almost pure noise near final diffusion step.

Figure 1 illustrates forward diffusion process.

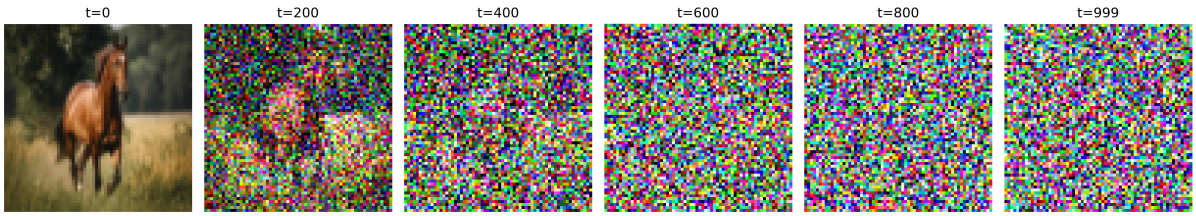


Figure 1: Forward diffusion process. Gaussian noise is gradually added until the original image becomes almost completely random.

Visualization confirms that implemented noise scheduler correctly follows diffusion process. At lower timesteps original image is still recognizable whereas at higher timesteps almost all the semantic information disappears.

5.2 Training Performance

Model was trained finally for the total 100 epochs using Adam optimizer with Mean Squared Error (MSE) loss. During training, loss decreased rapidly during first few epochs and then gradually converged. Best training loss achieved was approximately 0.02, indicating that model successfully learned to predict Gaussian noise added during forward diffusion process.

Figure 2 shows training loss obtained during final experiment.

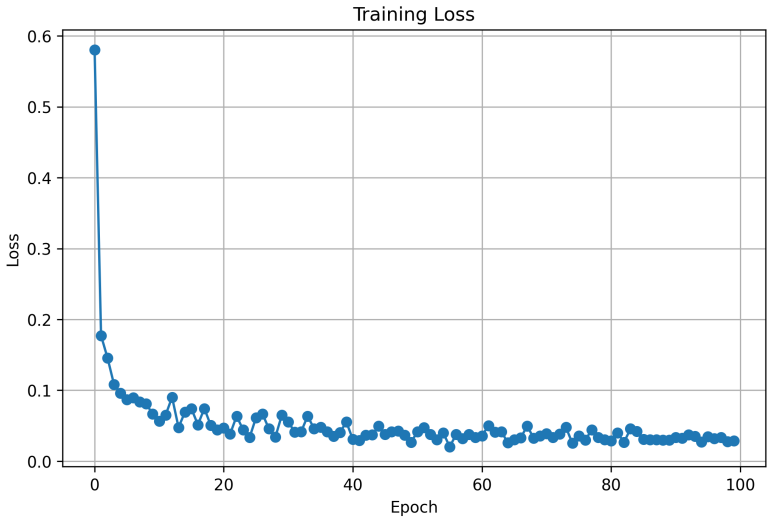


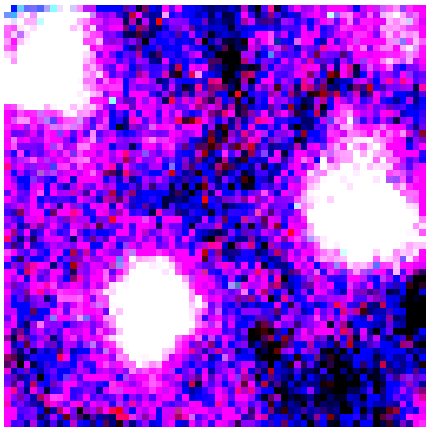
Figure 2: Training loss over 100 epochs. Loss decreases rapidly during early stages of training & gradually converges.

Overall training curve is smooth without any significant instability, showing that selected hyperparameters resulted in the stable optimization.

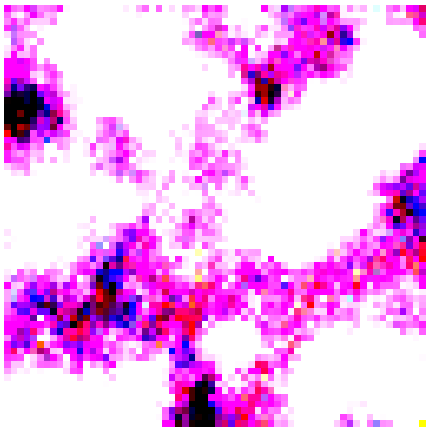
5.3 Generated Images

After training, best saved model was loaded & used to generate the new images by starting from random Gaussian noise, here since model is unconditional, every generation starts from a different random noise sample, producing different outputs even though same trained model is used.

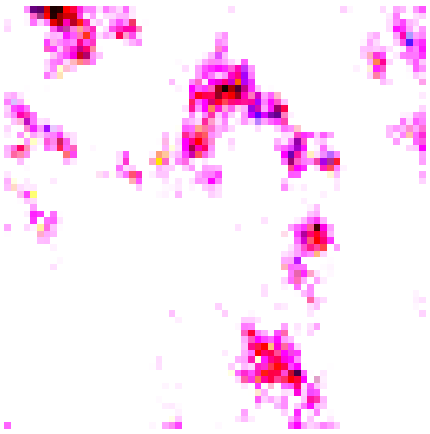
Figure 3 showing five generated samples obtained during the inference.



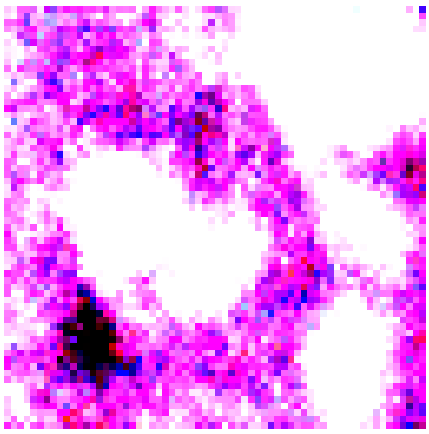
(a) Generated Sample 1



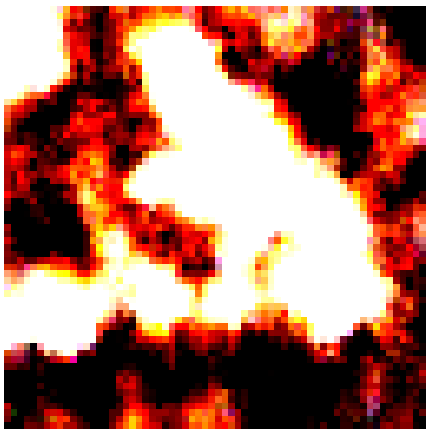
(b) Generated Sample 2



(c) Generated Sample 3



(d) Generated Sample 4



(e) Combined view of generated samples

Figure 3: Images generated by trained diffusion model from different random noise inputs.

Generated samples are different because each image starts from a unique random Gaussian noise distribution, and although model was trained on same dataset, randomness in initial noise results in different outputs after reverse diffusion process.

Generated images exhibit similar color distributions, textures, and coarse visual structures learned from animal dataset. Some samples contain stronger structural patterns, while others remain more abstract, and I expected this variation as a compact diffusion model trained on a relatively small dataset.

So, overall the generated samples demonstrate that trained model has learned denoising process and is capable of transforming the random noise into the structured images rather than just producing purely random the pixel values.

5.4 Reverse Diffusion Visualization

To better understand image generation process, I recorded the intermediate outputs from reverse diffusion during the inference.

Figure 4 shows how generated image gradually evolves from random gaussian noise into final output.

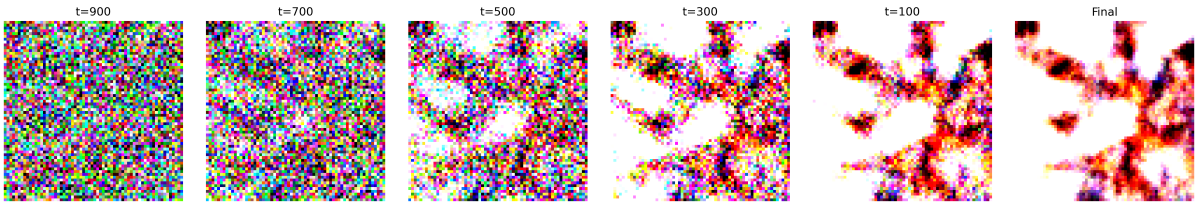


Figure 4: Reverse diffusion process showing gradual denoising from random noise to the final generated image.

Initially sample consists entirely of random noise, as reverse diffusion proceeds, coarse structures begin to emerge, so during final denoising steps, image becomes smoother, and more organized, thus illustrating how trained U-Net progressively removes the noise at each timestep.

6 Conclusion

Experimental results showed that training loss consistently decreased throughout training, thus indicating successful learning of denoising objective. Generated images, and reverse diffusion visualization further verified that implemented model correctly performs the diffusion process. Although image quality remains limited by compact network architecture, small dataset, and low image resolution, this successfully demonstrates fundamental principles of diffusion based image generation.